Programación de Inteligencia Artificial

Francisco Javier Sueza Rodríguez

13 de octubre de 2026

Centro:	IES Zaidin-Vergeles
Curso de Especialización:	Inteligencia Artificial y Big Data
Asignatura:	Programación de Inteligencia Artificial

Índice

1 Introducción a la Programación de Inteligencia Artificial **4**
1.1 Introducción a la Programación 4
1.1.1 Clasificación de los Lenguajes de Programación 5
1.1.2 Ciclo de Desarrollo del Software 7

Referencias **8**

Índice de figuras

Índice de tablas

1. Introducción a la Programación de Inteligencia Artificial

En esta era tecnológica, la Inteligencia Artificial está en todas partes transformando radicalmente como vivimos. Desde la automatización de proceso, hasta la toma de decisiones complejas, la IA se esta convirtiendo en un pilar fundamental en diversos campos. En este primer tema, exploraremos los fundamentos de programación de la IA, desde conceptos básicos de programación hasta consideraciónes propias de la IA.

También se presentarán los distintos perfiles profesionales relacionados con el desarrollo de la IA, así como las consideraciones éticas en su desarrollo y uso.

1.1. Introducción a la Programación

La **programación** es el proceso de diseñar un algoritmo pare resolver un problema concreto y traducirlo a instrucciones que un computador pueda interpretar y ejecutar. Un **algoritmo** es una secuencia lógica de instrucción diseñada para resolver un problema concreto, de manera efectiva, eficiente y precisa.

Un **buen programador** debe tener ciertos conocimientos básicos y reunir ciertas características para desarrollar su trabajo de forma eficiente. Entre estos conocimientos y características podemos encontrar los siguientes:

- **Capacidad de Abstracción**: es una habilidad básica para todo programador y consiste en la separación de un problema de su contexto para analizarlo y dividirlo en problemas más básicos para su resolución, encontrando similitudes con otros problemas durante el proceso y reutilizando soluciones independientemente del contexto si es necesario.
- **Pensamiento Lógico**: es la capacidad de encontrar relaciones entre objetos, reales o abstractos, y llegar a conclusiones basándose en principios y reglas. De una forma práctica, es la capacidad para desarrollar algoritmos eficientes para resolver problemas concretos.
- **Creatividad y Simplicidad**: un programador debe ser capaz de encontrar soluciones innovadores y eficientes para los desafíos planteados, manteniendo la simplicidad en las soluciones implementadas, lo que facilitará la comprensión y el mantenimiento del programa.
- **Conocimientos Técnicos**: un buen programador tendrá que conocer diferentes lenguajes de programación, así como tener conocimientos sobre estructuras de datos y algoritmos, así como el dominio de entornos de desarrollo y plataformas en la nube.
- **Habilidad para Depurar Errores**: es la capacidad para asilar, identificar y corregir los errores en el código de forma sistemática y eficiente.
- **Conocimientos Matemáticos**: en del desarrollo de software de inteligencia artificial es imprescindible tener sólidos fundamentos en lógica, álgebra y estadística:
    - **Lógica Matemática**: proporciona las reglas para determinar la validez de argumentos y se aplica en programación
    - **Álgebra**: trabaja con estructuras numéricas y es esencial para realizar operaciones en técnicas de aprendizaje automático.
    - **Estadística**: se encarga de organizar y analizar datos, antes de aplicar operaciones avanzadas en aprendizaje automático.

- **Capacidad de Trabajo en Equipo**: habilidad para colaborar con otros programadores, estableciendo canales de comunicación adecuados, sabiendo defender propuestas y reconociendo el talento. Todo ello articulado gracias al uso de **software de control de versiones**.

1.1.1. Clasificación de los Lenguajes de Programación

Un **lenguaje de programación** es un conjunto de reglas y sintaxis que permite a los programadores comunicarse con un ordenador para darle instrucciones y realizar tareas específicas. Esta compuesto de un conjunto de palabras clave, estructuras y convenciones que permiten implementar programas de software de forma organizada y eficiente.

Los lenguajes de programación se utilizan para construir todo de tipo de aplicaciones y soluciones tecnológicas. Cada lenguaje de programación tiene sus características, ventajas y desventaja, pudiendo ser utilizados para resolver diferentes tipos de problemas y desarrollar diferentes tipos de aplicaciones.

Los lenguajes de programación de pueden **clasificar** atendiendo a una variedad de propiedades y características que podemos resumir en las siguientes:

1. **Nivel de Abstracción**:
   - **Lenguajes de Bajo Nivel**: son aquellos cuya sintaxis está mas cercana al lenguaje máquina y son muy dependientes del hardware donde se ejecutará el programa. Algunos ejemplos pueden ser **MIPS** y **Motorola 68000**, aunque también hay otros lenguajes más conocidos como **C** o **C++**.
   - **Lenguajes de Alto Nivel**: son lenguajes cuyo sintaxis se parece más al lenguaje natural que al lenguaje máquina, ofreciendo un nivel mayor de abstracción, lo que los hace más fáciles de entender y usar. Permiten un desarrollo más rápido a costa de menor eficiencia. Algunos ejemplos de estos lenguajes son **Python**, **Java** o **Javascript**.
2. **Modo de Ejecución**:
   - **Lenguajes Compilados**: el código fuente escrito en estos programas se traduce a lenguaje máquina antes de su ejecución. Este proceso de traducción se realiza mediante un compilador, que genera un archivo ejecutable que puede ser ejecutado directamente por la computadora. Gracias a este proceso, estos lenguajes de programación suelen tener un tiempo de ejecución más rápido y mejor rendimiento. Algunos ejemplos son **C**, **C++**, **Rust** y **Swift**.
   - **Lenguajes Interpretados**: en estos lenguajes el código fuente es interpretado y ejecutado línea por línea mediante un intérprete en tiempo de ejecución. El interprete lee y ejecuta el código sin necesidad de generar un ejecutable. Este tipo de lenguajes dependen menos del hardware donde se ejecutan, siendo más portables y flexibles. Algunos lenguajes interpretados son **Python**, **R**, **Perl** o **PHP**.
   - **Lenguajes Híbridos**: algunos lenguajes pueden reunir características de estos dos tipos. Por ejemplo, **Java** se compila a un código intermedio llamado bytecode que posteriormente se ejecuta en la máquina virtual (JVM).
3. **Paradigma de Programación**:
   - **Lenguajes de Programación Imperativa**: estos lenguajes se centran en describir como debe realizarse las operaciones paso a paso, como **C** o **Python**.

- **Lenguajes de Programación Declarativa**: estos lenguajes se centran en describir que debe hacerse, sin especificar el cómo. El mayor ejemplo de este paradigma es **Prolog**, y aunque no sean lenguajes de programación, **SQL**, **CSS** o **HTML**.
- **Lenguajes de Programación Orientada a Objetos**: estos lenguajes organizan el software en objetos, que son unidades que pueden contener los datos y métodos. Pueden usar características como la herencia o el polimorfismo. Algunos ejemplos son **Java**, **C++** o **Ruby**.
- **Lenguajes de Programación Orientada a Eventos**: estos lenguajes están basados en el manejo de eventos y en la respuesta a acciones que lleva a cabo el usuario, por ejemplo como **VistualBasic**.
- **Lenguajes de Programación Funcional**: es un caso específico de la programación declarativa basado en el uso de verdaderas funciones matemáticas y que hace use intensivo del cálculo lambda [1]. Algunos ejemplos son **Haskell**, **Lisp** o **Erlang**.
- **Lenguajes Multiparadigma**: son lenguajes que pueden admiten múltiples paradigmas de programación, dejando que sea el programador que decida cual es el enfoque más adecuado. Ejemplos son **Python**, **Javascript** o **C++**

4. **Propósito del Lenguaje**:
   - **Lenguajes de Uso Específico**: son lenguajes que están diseñados para propósitos específicos, como le procesamiento de datos (**R**) o el desarrollo web (**CSS/HTML**).
   - **Lenguajes de Uso General**: son lenguajes que se pueden emplear para desarrollar diferentes tipos de aplicaciones, como **Java**, **Python** o **C**.
5. **Sistema de Tipado**:
   - **Lenguajes de Tipado Estático**: en estos lenguajes el tipo de cada variables debe ser declarado explícitamente y verificado antes de que el programa se ejecute, verificándose dichos tipos en tiempo de compilación. Esto hace que estos lenguajes sean más robustos y seguros. Algunos ejemplos son **C** o **Java**.
   - **Lenguajes de Tipado Dinámico**: en estos lenguajes la variables pueden tomar cualquier valor tipo de dato en cualquier momento, pudiendo cambiar de tipo durante la ejecución, ya que los tipos se verifican en tiempo de ejecución. Este tipo de lenguajes ofrecen mayor simplicidad y flexibilidad en el desarrollo. Algunos ejemplos son **Python**, **Ruby** o **Javascript**.
6. **Rigidez del Tipado**:
   - **Lenguajes de Tipado Fuerte**: los lenguajes de tipado fuerte establecen fuertes restricciones sobre como se pueden hacer la conversión de tipos y se requiere una conciencia exacta de los tipos para las operaciones. Son ejemplos **Java** o **C** donde las conversiones de tipo deben declararse explícitamente.
   - **Lenguajes de Tipado Débil**: en los lenguajes de tipado débil las conversiones entre tipos son menos estrictas, permitiendo la conversión implícita entre diferentes tipos de datos y la conversión automática en determinadas circunstancias. Algunos ejemplos son **Javascript** o **PHP**.

Como vemos, hay una gran variedad de lenguajes de programación cada uno con sus características. La **elección de un lenguaje** es un aspecto crucial para que un proyecto llegue a buen termino y que dependerá, entre otros factores, de al naturaleza del proyecto, la experiencia del equipo o de la plataforma donde se implantará el software.

1.1.2. Ciclo de Desarrollo del Software

El **desarrollo de software** es el proceso de crear aplicaciones informáticas utilizando un proceso sistemático. En un campo fundamental de la informática que abarca diversas actividades orientadas a construir soluciones informáticas que satisfagan las necesidades específicas de los usuarios u organizaciones.

Este proceso, se compone de **diferentes etapas** que vamos a pasar a enumerar a continuación:

1. **Definición del Problema**: esta primera fase consiste en definir, de forma clara y precisa, el problema que se pretende resolver, definiendo el marco del problema y los objetivos a alcanzar.
2. **Análisis de Requisitos**: en esta fase de identifican y documentan los requisitos del software. Esto implica conocer las necesidades de los usuarios finales y definir las funcionalidades y características que debe tener el software. Se creará un listado exhaustivo con todos los requisitos que representa un contrato entre cliente y desarrollador, a la vez que permite al cliente que lo revise y valide, antes de pasar a otras fase.
3. **Planificación del Desarrollo**: consiste en la planificación del resto de fases del proceso de desarrollo. Se identifican las tareas a realizar, teniendo en cuenta las dependencias entre tareas y se estiman los tiempos de desarrollo, identificando los recursos y personal necesarios. Esta fase se apoya en **Diagramas de Pert** y **Gantt**.

Referencias

[1] Wikipedia - Programación Funcional.
https://es.wikipedia.org/wiki/Programaci%C3%B3n_funcional.